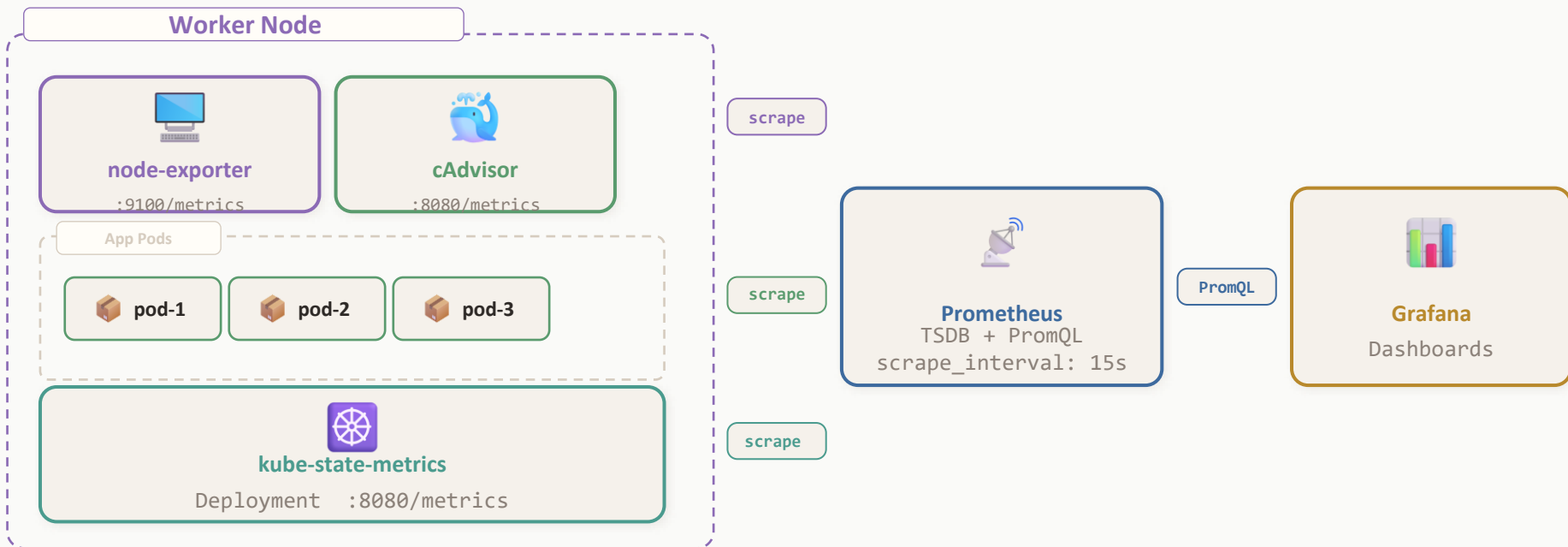


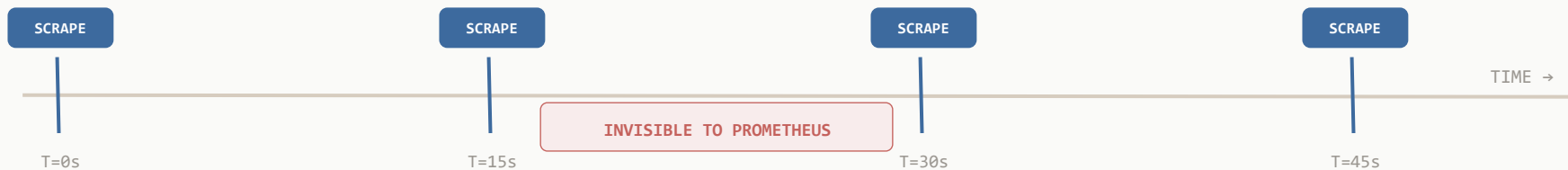
Hard Lessons Learned from Prometheus Exporters: Observability Challenges in Kubernetes Environments

HungWei Chiu @ 2026/04/23
Observability Day

Prometheus in Kubernetes: Scrape Topology



Pull Model: The Sampling Contract You Implicitly Signed



Metrics Tell You What Happened — Not Why



WHAT HAPPENED

container_cpu_throttled_seconds_total ↑

http_requests_total latency p99 ↑

node_memory_MemAvailable_bytes ↓

GAP



WHY IT HAPPENED

? Noisy neighbour on the same node?

? Node under memory pressure (PSI)?

? CFS bandwidth throttle from wrong CPU limit?

A good K8s administrator should focus on resolving the root cause to mitigate alerts, rather than simply adding more alerts.

Three Personas, Three Observability Planes



App Owner

"Is my service healthy?"

Watches

- Request rate, error rate, p99 latency
- DB query time, cache hit ratio
- Downstream service health



Node-level pressure is invisible



Service SRE

"Is my SLO burning?"

Watches

- Pod availability, restart rate
- CPU/memory requests vs limits
- HPA scaling, deployment health



Platform incidents look like app failures



Platform Admin

"Is the cluster healthy?"

Watches

- Node conditions, disk/memory pressure
- Pod evictions, OOM events
- Scheduler queue depth, bind latency



Service metrics don't surface here

App-Owner Lenses: RED · Four Golden Signals (+ USE for Infra Context)



RED

Rate · Errors · Duration

PRIMARY — App Owner / Service Engineer

Rate

Requests per second served by the service

Errors

Fraction of requests that are failing

Duration

Distribution of time each request takes

💡 Start here for any HTTP microservice



Four Golden Signals

Latency · Traffic · Errors · Saturation

PRIMARY — Service SRE / Google SRE

Latency

Time to serve a request (success vs failure)

Traffic

Demand on the system (RPS, QPS, streams)

Errors

Rate of requests that fail explicitly

Saturation

How full the service is (queue depth, lag)

💡 Google SRE Book — the standard SLO anchor



USE

Utilization · Saturation · Errors

INFRA CONTEXT — Platform / Node Owner

Utilization

% time resource is busy (CPU %, disk busy %)

Saturation

Extra work queued (run-queue length, mem swap)

Errors

Error events (hardware faults, dropped packets)

💡 Designed for resources, not services

AI can help us quickly retrieve and compare this data; the real challenge lies in not knowing what specific data to provide for analysis and comparison.

Case Study: etcd — The Control Plane Database

CASE 01 · etcd Defrag & Storage Pressure

1. SIGNAL

2. LIMIT

3. CONTEXT

What it is

etcd is the durable state machine for the entire cluster.
Slow fsync → leader elections → API server latency → cluster halts.

Symptom you see

Dashboard: API server latency rising. etcd DB size approaching 8 GiB quota. Leader changes spiking.

What Prometheus shows

`etcd_db_total_size_in_bytes` · `etcd_server_leader_changes_seen_total` ·
`etcd_disk_backend_commit_duration_seconds`

etcd: What Storage Pressure Looks Like in Prometheus

CASE 01 • etcd Defrag & Storage Pressure

1. SIGNAL

2. LIMIT

3. CONTEXT

```
etcd_db_total_size_in_bytes
```

DB at 80 % of 8 GiB quota

```
etcd_server_leader_changes_seen_total
```

Frequent leader elections — cluster instability



The alert tells you **WHAT** is full — not **WHY** it grew or whether it will recur.

etcd: Defrag Is a Pressure Release, Not a Root-Cause Fix

CASE 01 • etcd Defrag & Storage Pressure

1. SIGNAL

2. LIMIT

3. CONTEXT

Defrag treats the symptom — it does not explain why etcd grew.

✓ Standard runbook

1. Alert fires: DB > 80 %

2. Run: `etcdctl defrag --cluster`

3. Alert clears — ticket closed

✗ Still unknown

? Which resource type consumed the space?

? Will the alert fire again next week?

💡 *Defrag is a pressure-release valve. You close the alert but leave the underlying problem open.*

etcd: The Context That Prometheus Cannot Provide

CASE 01 · etcd Defrag & Storage Pressure

1. SIGNAL

2. LIMIT

3. CONTEXT

Prometheus gives you

`etcd_db_total_size_in_bytes`

`etcd_mvcc_db_total_size_in_bytes`

`etcd_mvcc_db_total_size_in_use_in_bytes`

To diagnose you also need

Key-space breakdown by resource type

API audit logs (who writes most?)

Compaction frequency & revision lag

etcd: The Context That Prometheus Cannot Provide

CASE 01 · etcd Defrag & Storage Pressure

1. SIGNAL

2. LIMIT

3. CONTEXT

✓ Potential Root Causes

- ✓ Overside objects (e.g., ConfigMaps, Secrets, CR)
- ✓ High-Frequency Object update (Operator)
- ✓ Rapid Pod Churn (CronJobs, Airflow...etc)
- ✓ Unused replicaset revision history

🔥 Root cause needs key-space data, audit logs, and compaction metrics — none from standard Prometheus exporters.

etcd: The Context That Prometheus Cannot Provide

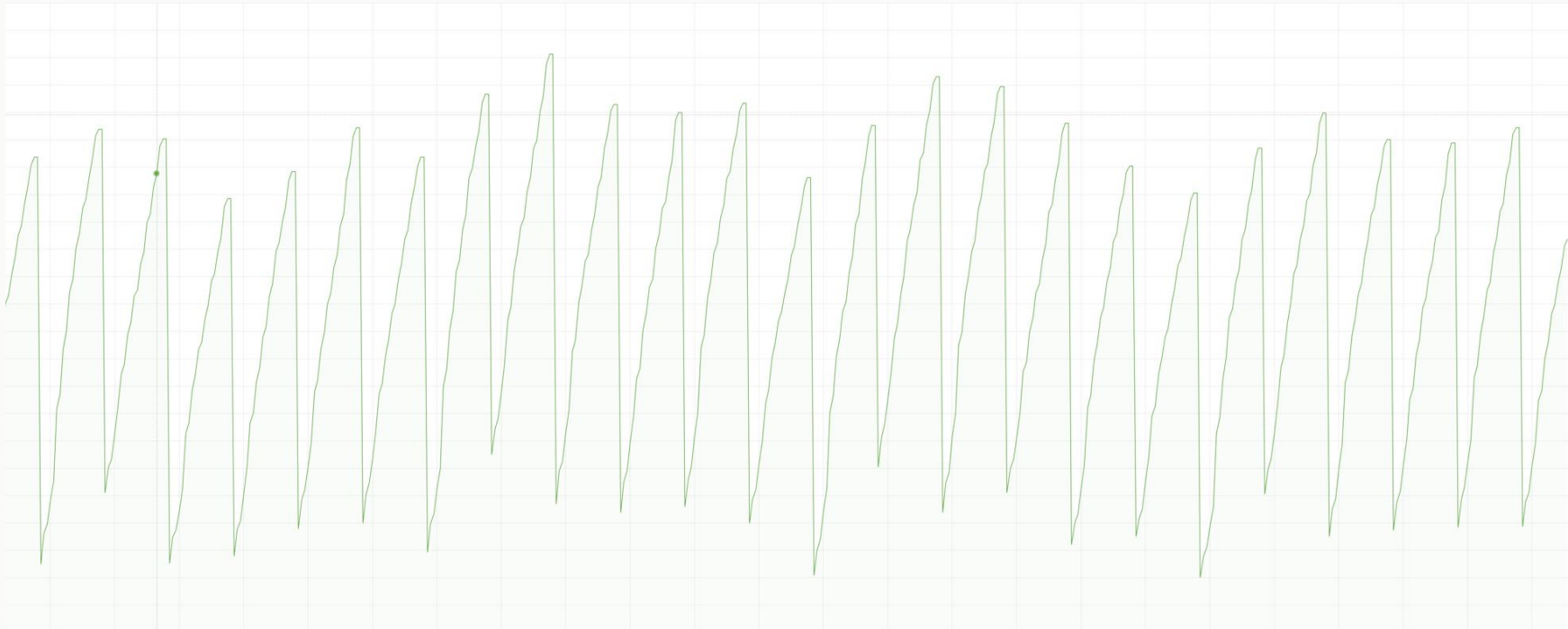
CASE 01 • etcd Defrag & Storage Pressure

1. SIGNAL

2. LIMIT

3. CONTEXT

API Server compact every 5 mins



etcd: The Context That Prometheus Cannot Provide

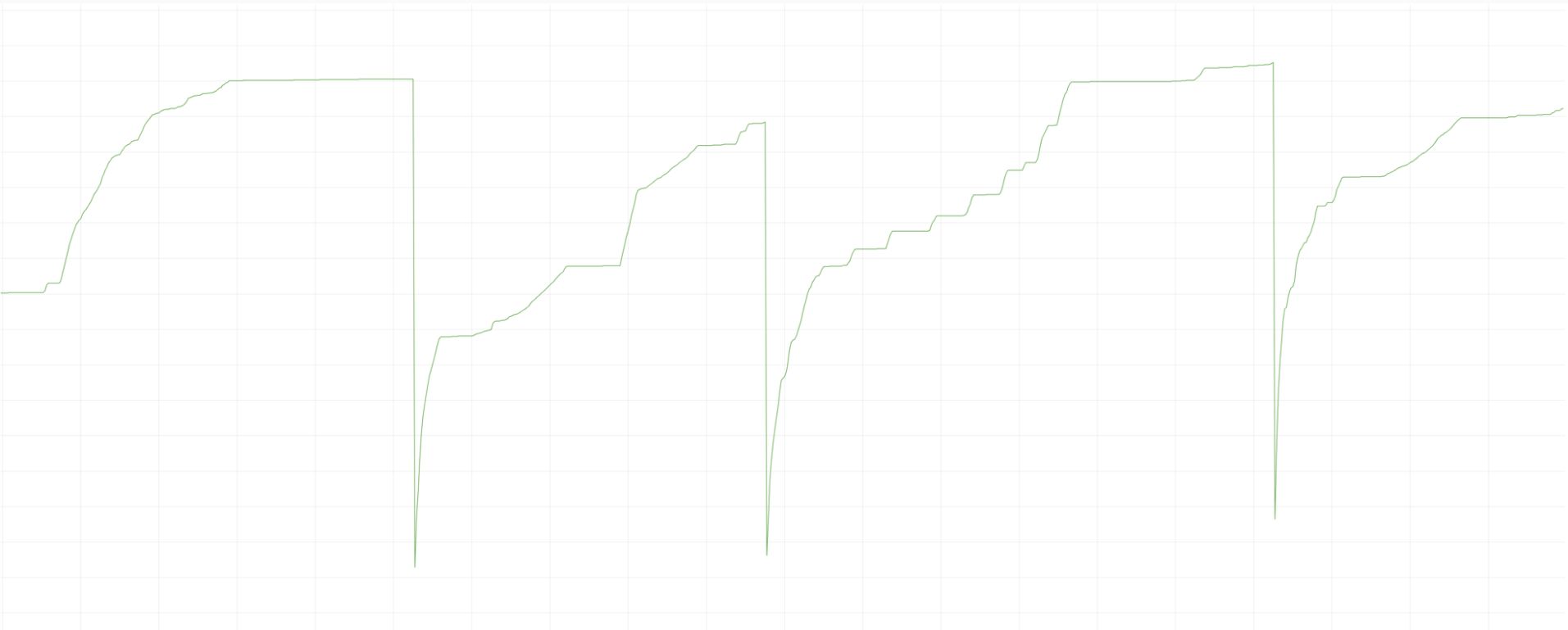
CASE 01 · etcd Defrag & Storage Pressure

1. SIGNAL

2. LIMIT

3. CONTEXT

Defrag regularly



etcd: The Context That Prometheus Cannot Provide

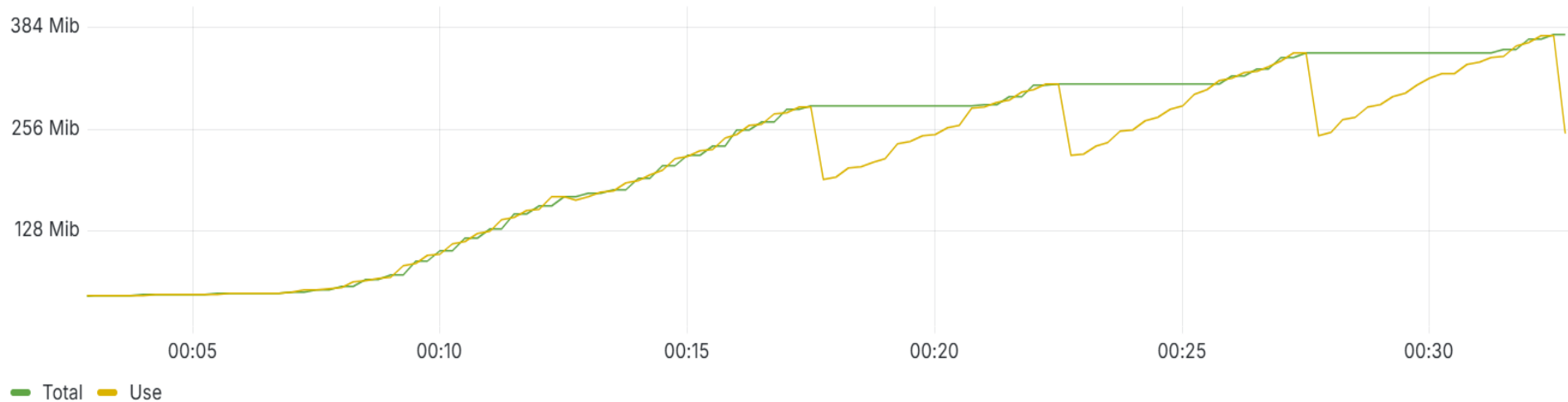
CASE 01 · etcd Defrag & Storage Pressure

1. SIGNAL

2. LIMIT

3. CONTEXT

- ✓ Cronjobs (700 pods per 3 minutes)
- ✓ Pod: Sleep 10 seconds then completed



etcd: The Context That Prometheus Cannot Provide

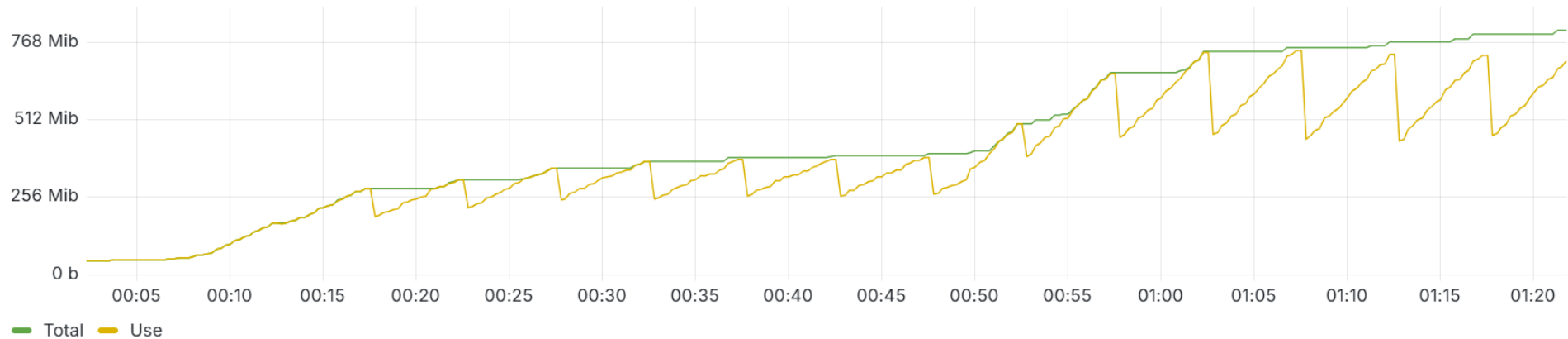
CASE 01 · etcd Defrag & Storage Pressure

1. SIGNAL

2. LIMIT

3. CONTEXT

- ✓ 10 * ConfigMap (1MiB)
- ✓ Update ConfigMap label every minutes



etcd: The Context That Prometheus Cannot Provide

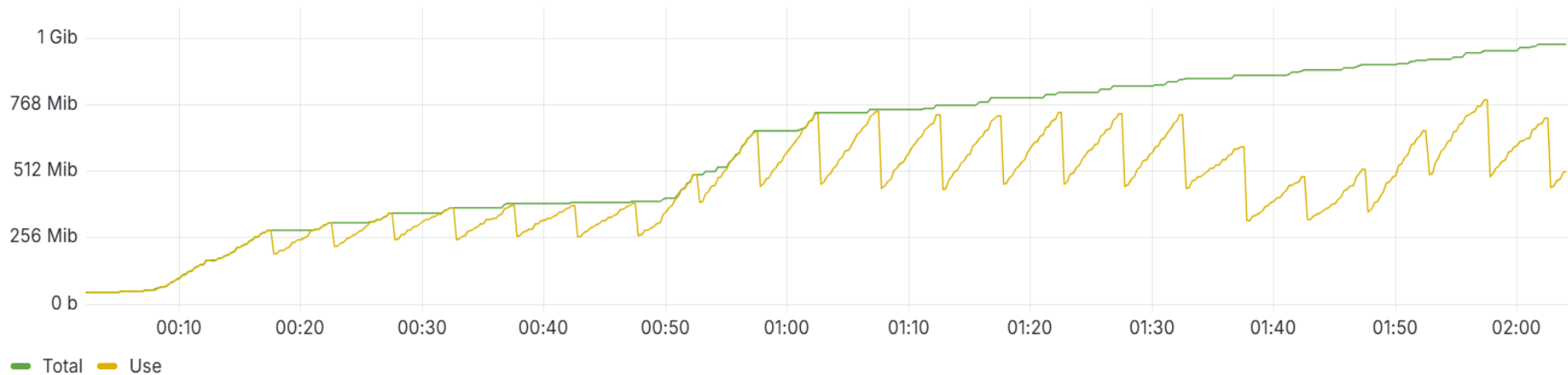
CASE 01 • etcd Defrag & Storage Pressure

1. SIGNAL

2. LIMIT

3. CONTEXT

- ✓ 500 * Deployment with HPA
- ✓ Update HPA objects periodically



etcd: The Context That Prometheus Cannot Provide

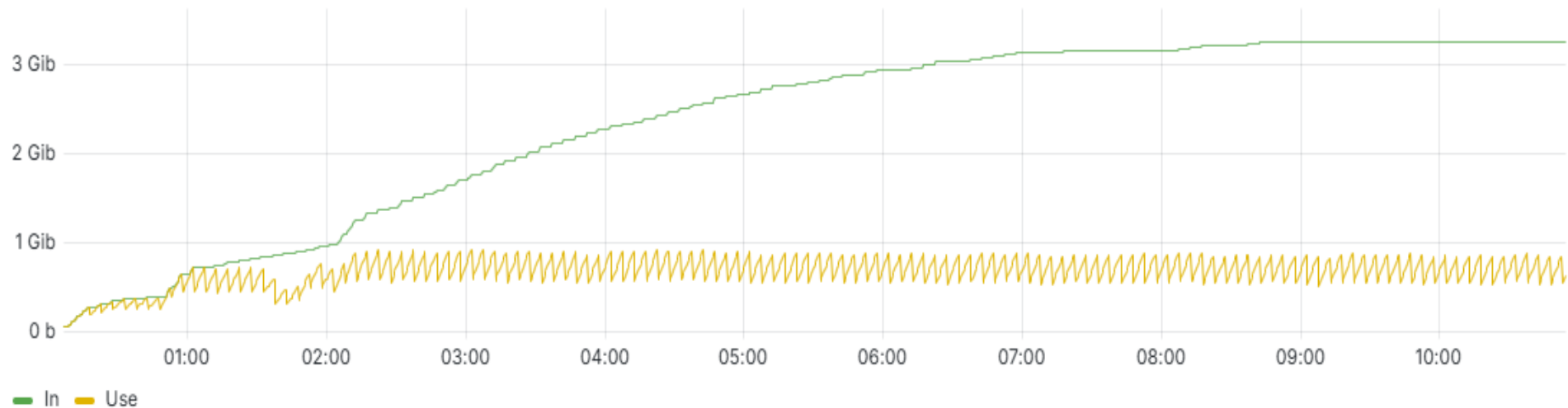
CASE 01 · etcd Defrag & Storage Pressure

1. SIGNAL

2. LIMIT

3. CONTEXT

- ✓ 10 * ConfigMap (1MiB)
- ✓ Update ConfigMap label every 10 seconds



etcd: The Context That Prometheus Cannot Provide

<https://github.com/etcd-io/etcd/issues/19806>

<https://github.com/etcd-io/bbolt/issues/694>

Performance improvement ideas for v3.7 #19806

Open



serathius opened on Apr 26, 2025 · edited by serathius

Edits

What would you like to be added?

Want to collect ideas for improvements with large impact. While v3.6 was targeting stability and testing, the v3.7 should show that there is still a lot of place for performance improvement in etcd. Goal is to inspire more ambitious work working towards it in v3.7. This is not exclusive list, and I'm happy to add ideas:

- **RangeStream** - Issue [Open](#) [Large range queries from api-server could take down etcd due to OOM #12342](#) Bring the etcd. More about it in <https://www.youtube.com/watch?v=SdLLOcNZNSE>. Based on my initial research we cannot implement streaming encoders for GRPC, so the only option seems like adding separate streaming range method. PoC in [Implementation of streaming range request #19766](#) is very promising. Older attempt [Yxjetcd rangestream #12342](#)
- **Async raft** - Bring async raft improvements in [raft: support asynchronous storage writes raft#8](#) to etcd. Issue [raft #18027](#), Draft implementation
- **Reduce memory usage of etcd member catchup mechanism** - Introduce a mechanism to propagate a slowest member index, so other members can compact raft log based on it. [Reduce memory usage of etcd member catchup mechanism #17098](#)

Incremental etcd defrag #694

Open



serathius opened on Feb 13, 2024 · edited by serathius

Edits Member ...

Resurrecting old [@ptabor](#) idea that we had discussed some time ago but didn't have time to implement.

Problem

- Bbolt page management still works like Windows 95. To be efficient and it requires to be periodically defragmented by calling a bbolt compact function. This is problematic from both reliability and maintenance perspective. Defragmenting etcd blocks writes, which in large clusters can take in tens of seconds. In high churn clusters users need to manage a separate process that periodically defragments etcd

Goal

- Reduce the need for heavy defragment operation by doing small incremental work during transactions. This should allow us to maintain good enough fragmentation level without a need for a periodic "stop the world" operation.

Case Study: Zombie Processes as a Platform Health Signal

CASE 02 • Zombie Processes

1. SIGNAL

2. LIMIT

3. CONTEXT

What it is

Zombie processes are defunct — they finished but their parent never called `wait()`. They hold a PID slot. Rapid accumulation signals a parent process misbehaving.

Signal you watch

`node_processes_state{state="Z"}` rising steadily — especially past 10-15 — is a reliable early warning of a stuck or crashing containerized parent.

Why Prometheus can't tell you more

`node-exporter` counts zombies but cannot identify which pod or containers is the source. You need process-level tooling (`ps`, `/proc`) to close the loop.

 *`node_processes_state{state="Z"}` is a platform health signal, not an app metric. It lives in the infrastructure observation layer.*

Zombies: You Can Count Them — You Cannot Attribute Them

CASE 02 • Zombie Processes

1. SIGNAL

2. LIMIT

3. CONTEXT

node-exporter counts zombies — it cannot say whose they are.

✓ **node-exporter sees**

```
node_processes_state{state="Z"} = 14
```

```
instance = "node-03"
```

No pod or containers label

✗ **Operator still needs**

? Which pod is creating zombies?

? Which containers owns it?

? How long has this been occurring?

🔥 **node-exporter gives a node-level count with no pod or container label — you know there are zombies but not whose they are.**

Zombies: Root Causes and the Context That Closes the Gap

CASE 02 • Zombie Processes

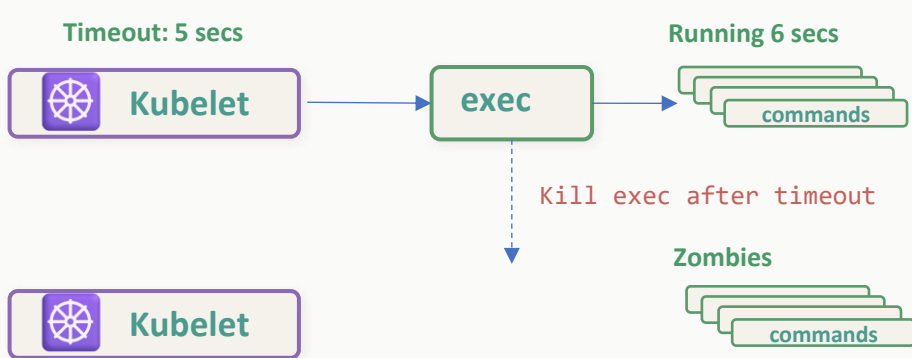
1. SIGNAL

2. LIMIT

3. CONTEXT

✓ Potential Root Causes

✓ Improper probe timeouts may lead to silent zombie processes.



```
level=warning msg="Found defunct process with PID 2873360 (timeout)"
level=warning msg="Found defunct process with PID 2879917 (timeout)"
level=warning msg="Found defunct process with PID 2879921 (grep)"
level=warning msg="Found defunct process with PID 2880004 (timeout)"
level=warning msg="Found defunct process with PID 2880005 (grep)"
level=warning msg="Found defunct process with PID 2886711 (timeout)"
level=warning msg="Found defunct process with PID 2886715 (grep)"
level=warning msg="Found defunct process with PID 2886725 (timeout)"
level=warning msg="Found defunct process with PID 2886726 (grep)"
level=warning msg="Found defunct process with PID 2887578 (timeout)"
level=warning msg="Found defunct process with PID 2887579 (grep)"
level=warning msg="Found defunct process with PID 2894550 (timeout)"
level=warning msg="Found defunct process with PID 2894551 (grep)"
level=warning msg="Found defunct process with PID 2897362 (timeout)"
level=warning msg="Found defunct process with PID 2897363 (timeout)"
```

Case Study: Noisy Neighbor — A Structural Scheduling Problem

CASE 03 • Noisy Neighbor

1. SIGNAL

2. LIMIT

3. CONTEXT

The setup

Multiple teams deploy onto shared nodes. Kubernetes isolates by cgroup limits, but the kernel does not isolate page cache, IRQ, network backplane, or disk I/O by default.

What you see

Team A's service latency spikes. CPU is at 35 %. Memory is fine. Nothing in Team A's metrics explains it. But something on the same node is consuming shared resources.

Why it's hard to find

node-exporter reports aggregate node metrics. cAdvisor reports per-container usage. Neither shows cross-tenant resource contention directly.

 Noisy neighbor is a structural problem: the scheduler placed two competing teams on the same node. The next slides show what signals surface this.

Noisy Neighbor: Why CPU and Memory Dashboards Mislead You

CASE 03 • Noisy Neighbor

1. SIGNAL

2. LIMIT

3. CONTEXT

Dashboard green. Latency 5× above SLO.

✓ Your dashboard shows

container_cpu_usage_seconds_total

32 %

container_memory_working_set_bytes

420 Mi

✗ What CPU / Memory cannot show



Disk I/O bandwidth contention



Memory Reclaim Pressure



Kernel Scheduler Waiting Time

🔥 CPU 32 %, Memory 420 Mi, Pod Ready — all green. p99 latency 5× above SLO. Service metrics do not see shared kernel resource contention.

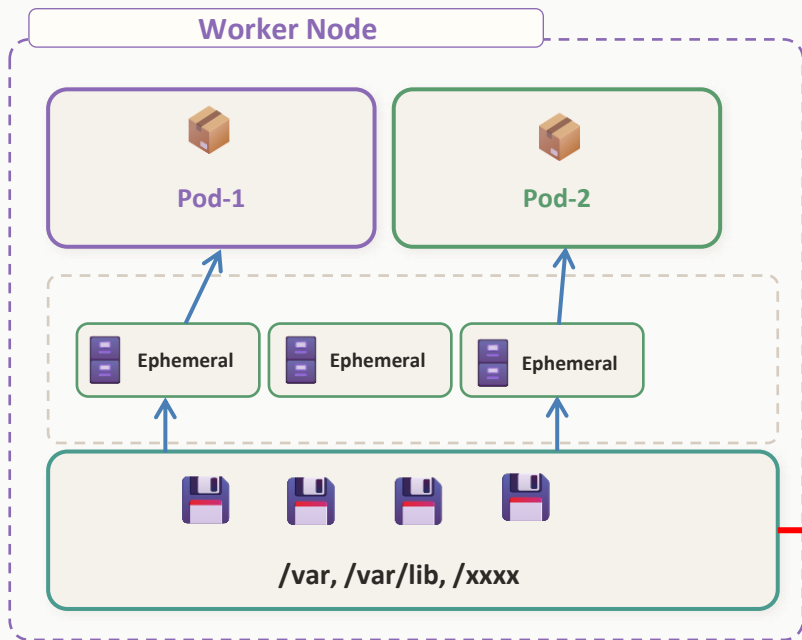
Noisy Neighbor: Signals That Go Beyond CPU and Memory

CASE 03 • Noisy Neighbor

1. SIGNAL

2. LIMIT

3. CONTEXT



✗ Kubelet Node Disk Pressure / Disk Performance

✗ System File overconsumption

✗ Container Image and Log Accumulation

✗ Application-Generated Ephemeral Files



KubeNodeDiskPressure Alerts

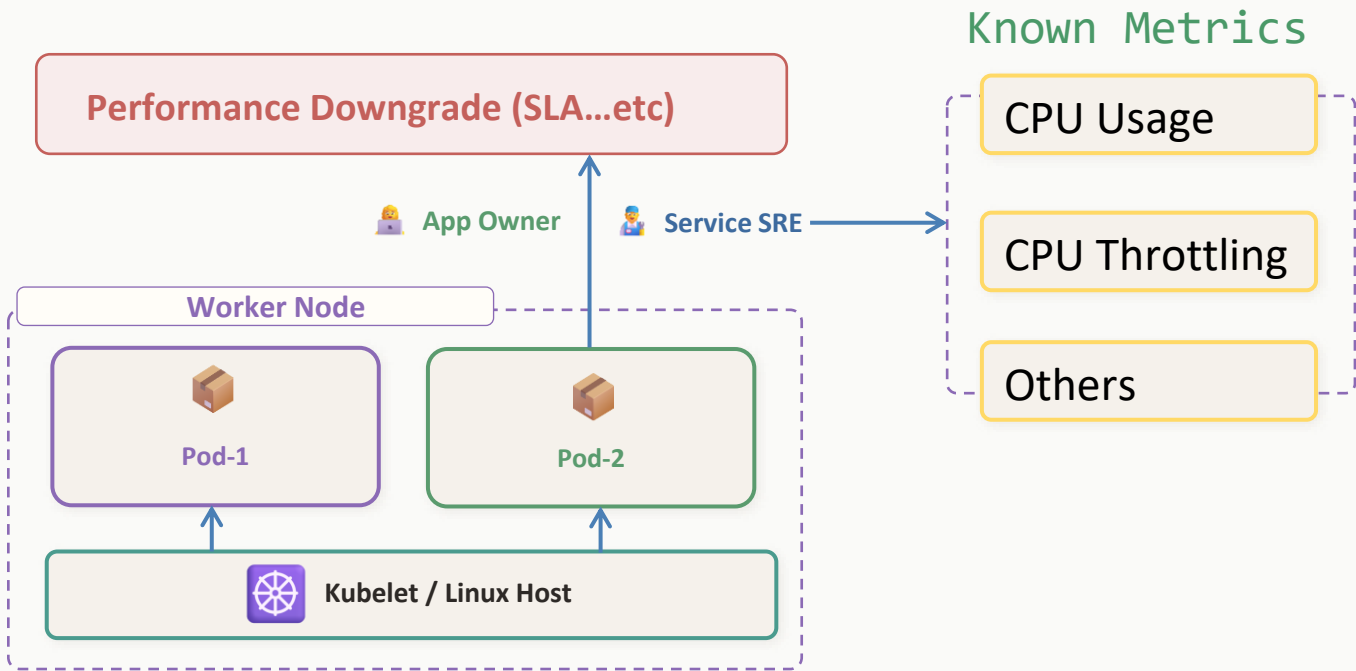
Noisy Neighbor: Signals That Go Beyond CPU and Memory

CASE 03 • Noisy Neighbor

1. SIGNAL

2. LIMIT

3. CONTEXT



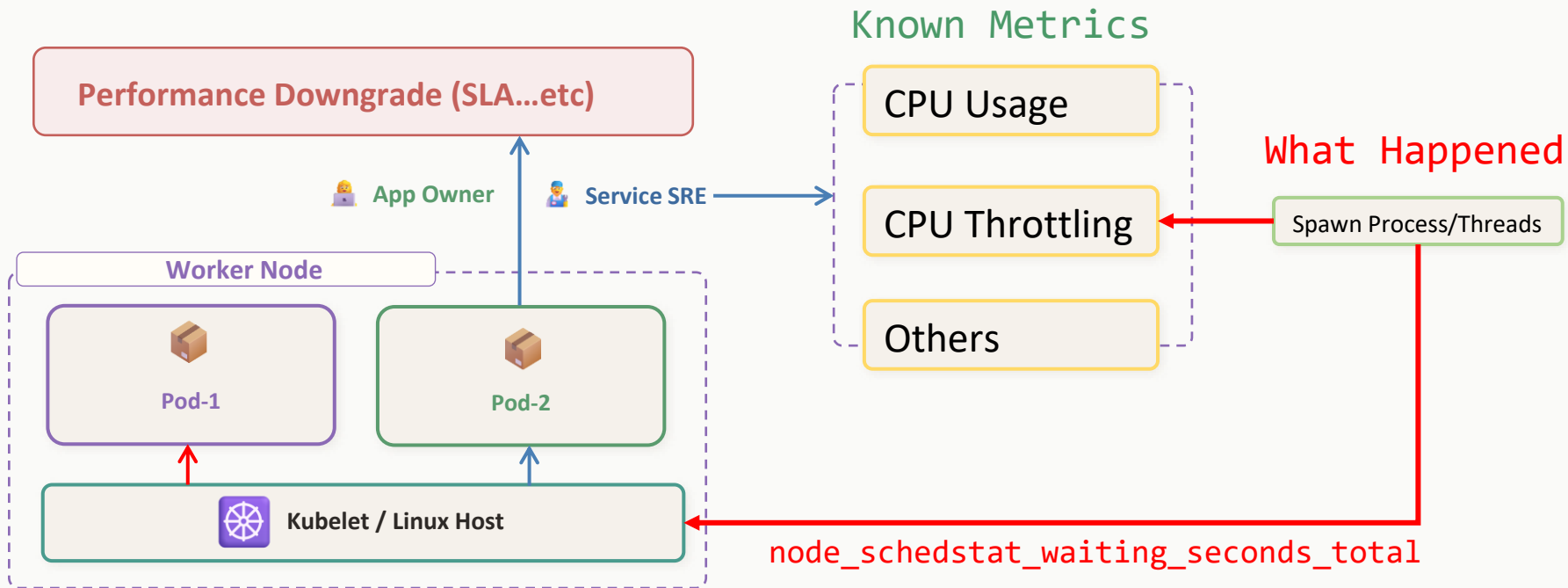
Noisy Neighbor: Signals That Go Beyond CPU and Memory

CASE 03 • Noisy Neighbor

1. SIGNAL

2. LIMIT

3. CONTEXT



Noisy Neighbor: Signals That Go Beyond CPU and Memory

CASE 03 • Noisy Neighbor

1. SIGNAL

2. LIMIT

3. CONTEXT

The Node Exporter itself can also be affected, leading to random loss of metrics and complicating troubleshooting efforts.



Noisy Neighbor: Signals That Go Beyond CPU and Memory

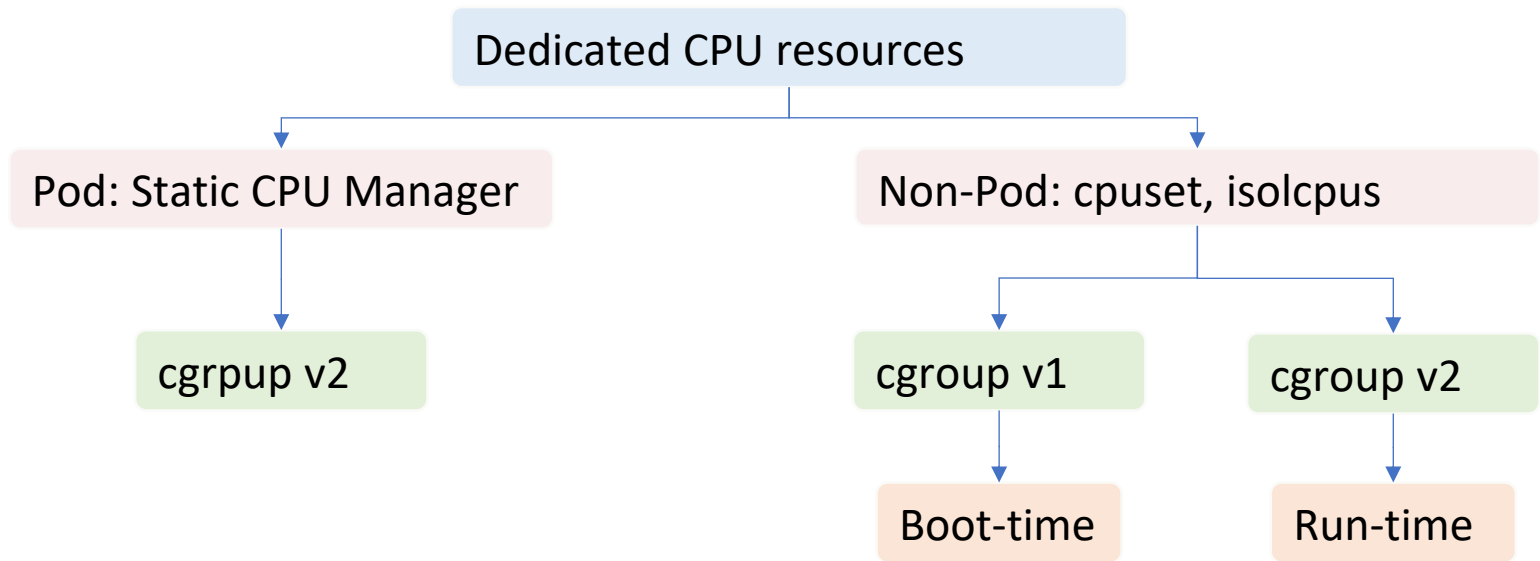
CASE 03 • Noisy Neighbor

1. SIGNAL

2. LIMIT

3. CONTEXT

In the worst-case scenario, this could even impact critical control plane daemons such as Kubelet, Kube-proxy, and CRI-O.



Noisy Neighbor: Signals That Go Beyond CPU and Memory

CASE 03 • Noisy Neighbor

1. SIGNAL

2. LIMIT

3. CONTEXT

<https://github.com/kubernetes/kubernetes/issues/115994>

<https://github.com/kubernetes/kubernetes/issues/87862>

containers get scheduled on --reserved-cpus #115994

🔒 Closed



iancoolidge opened on Feb 23, 2023 · edited by iancoolidge

What happened?

Node has 64 vCPUs. CPU manager policy is 'static'

8 vCPUs are marked as reserved (via --reserved-cpus)

The remaining 56 vCPUs are all isolated via kernel command line.

systemd.cpu_affinity is set for all of the --reserved-cpus also (I think this is redundant with isolated CPU

Workload has two types of pods: those that request full integer CPUs (we'll call these 'dedicated CPU t
'normal tasks')

result:

dedicated CPU tasks get correctly scheduled on the isolated vCPUs

normal tasks do not, and instead, get scheduled to the --reserved-cpus set.

isolcpus and --reserved-cpus don't work as expected #87862

🔒 Closed



pikrzysztof opened on Feb 6, 2020 · edited by pikrzysztof

Edits ▾ ⋮

Release notes from 1.17 says:

A new kubelet command line option, --reserved-cpus, is introduced to explicitly define the CPU list that will be reserved for system. For example, if --reserved-cpus=0,1,2,3 is specified, then cpu 0,1,2,3 will be reserved for the system. On a system with 24 CPUs, the user may specify isolcpus=4-23 for the kernel option and use CPU 4-23 for the user containers

I've got an 8-core, bare-metal machine running k8s 1.17.2 and kernel 5.3.0-24-generic (ubuntu).

I'm configuring kubelet like this:

```
reservedCpus: 0
kubeReserved:
  cpu: "100m"
  memory: 250Mi
systemReserved:
  cpu: "0.5"
  memory: "7Gi"
```



While containers offer lightweight virtualization, they do not provide complete isolation from resource contention. Consequently, "noisy neighbor" issues can impact other containers in ways that are not always evident through standard metrics.

Kernel Blind Spots: node-exporter Lives in User Space

✓ What user-space metrics tell you

- ✓ `node_cpu_seconds_total` (35 %)
- ✓ `node_memory_MemAvailable_bytes` (3.2 Gi)
- ✓ `node_disk_read_bytes_total` (nominal)
- ✓ `node_network_receive_bytes_total` (nominal)

✗ What stays hidden in kernel space

- ✗ kernel scheduler internals
- ✗ IRQ / softirq per CPU distribution
- ✗ page reclaim / kswapd pressure
- ✗ cgroup hierarchy throttle decisions

Kernel Forensics: Evidence Is Destroyed Before You Look

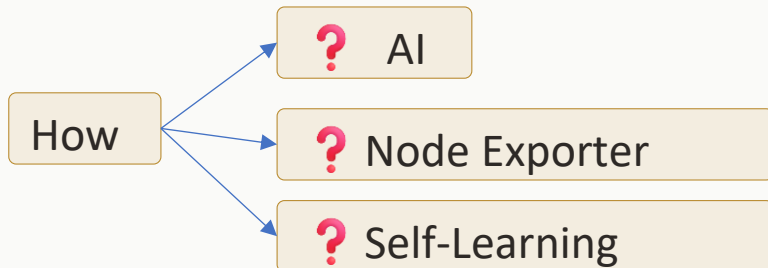
By the time you look, the forensic evidence is gone.

Stack traces / core dumps

Container runtimes discard core dumps by default. You get a crash timestamp — the call stack is gone.

Node reboot

A kernel panic clears /proc state and in-memory counters. Prometheus shows a counter reset with no explanation.



Usage $\neq$ Pressure.

CPU at 70 % utilisation can still starve workloads if one cgroup dominates the scheduler queue.
PSI measures the time tasks actually spent waiting — not how busy the resource appeared.



Usage Metrics

CPU: 0-100 % of allocated core time

Memory: `working_set_bytes` vs `limit`

Disk: bytes read/written per second

Problem: 70 % can still starve workloads



PSI (What You Need)

Measures: time tasks stalled waiting for a resource

some: ≥ 1 task stalled, others made progress

full: ALL tasks stalled, no progress for that period

Direct answer: 'Is this resource a bottleneck?'

Usage $\neq$ Pressure.

VAGUE OLD METRIC (e.g., Load Average)

RESOURCE



LOAD: high (e.g., 5.0)

?

TASKS/PROCESSES



Traditional metrics only show **waiting count**.

PSI: PRECISE STALL DIAGNOSTICS

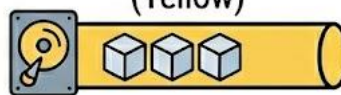
CPU STALL
(Green)



MEMORY STALL
(Red)



I/O STALL
(Yellow)



CONFIDENT SRE: Precise diagnosis.

Pod-Level PSI: Kubernetes 1.34 — Feature Gate KubeletPSI (Beta)

Before K8s 1.34 — Node-Level PSI Only

 /proc/pressure/ is a per-node kernel interface

 PSI values aggregate all cgroups on the node

 Cannot attribute pressure to a specific pod or namespace

 If Pod A starves Pod B, node PSI rises but you cannot tell who caused it

Kubernetes 1.34 — Pod-Level PSI (beta)

 Per-pod cgroup PSI files

 Prometheus labels: pod + namespace

 Noisy-neighbor attribution solved

 Requires cgroup v2 + kernel 4.20+

Takeaway: What to Add to Your Prometheus Setup

As much as the world pushes for AIOps, the most 'human' way to work in this AI era is still to invest time in understanding the 'why' to find a better 'how'.